

Aprendizado de máquina aplicado ao xadrez

Comparações entre Modelos ResNet e DQN

Daniel Couto | Aprendizado de Máquina 2 | IMPA

Sumário

- Introdução
- Engine
- CNN
- Resnet
- DQN
- Resultados

Introdução

Abordagem Inicial

Treinamento Supervisionado

Treinar um modelo preditivo utilizando um conjunto de dados massivo com posições de xadrez reais e suas respectivas avaliações numéricas atribuídas pelo Stockfish.

Aplicação Prática

Tomada de Decisão

Com o modelo consolidado para estimar o valor de qualquer estado do tabuleiro, o sistema calcula as ramificações de lances possíveis e escolhe a jogada ideal.

Em 1950, o matemático americano Claude Shannon estimou o número de partidas possíveis no xadrez:

$$35^{80} \approx 10^{120}$$

Dataset de Referência

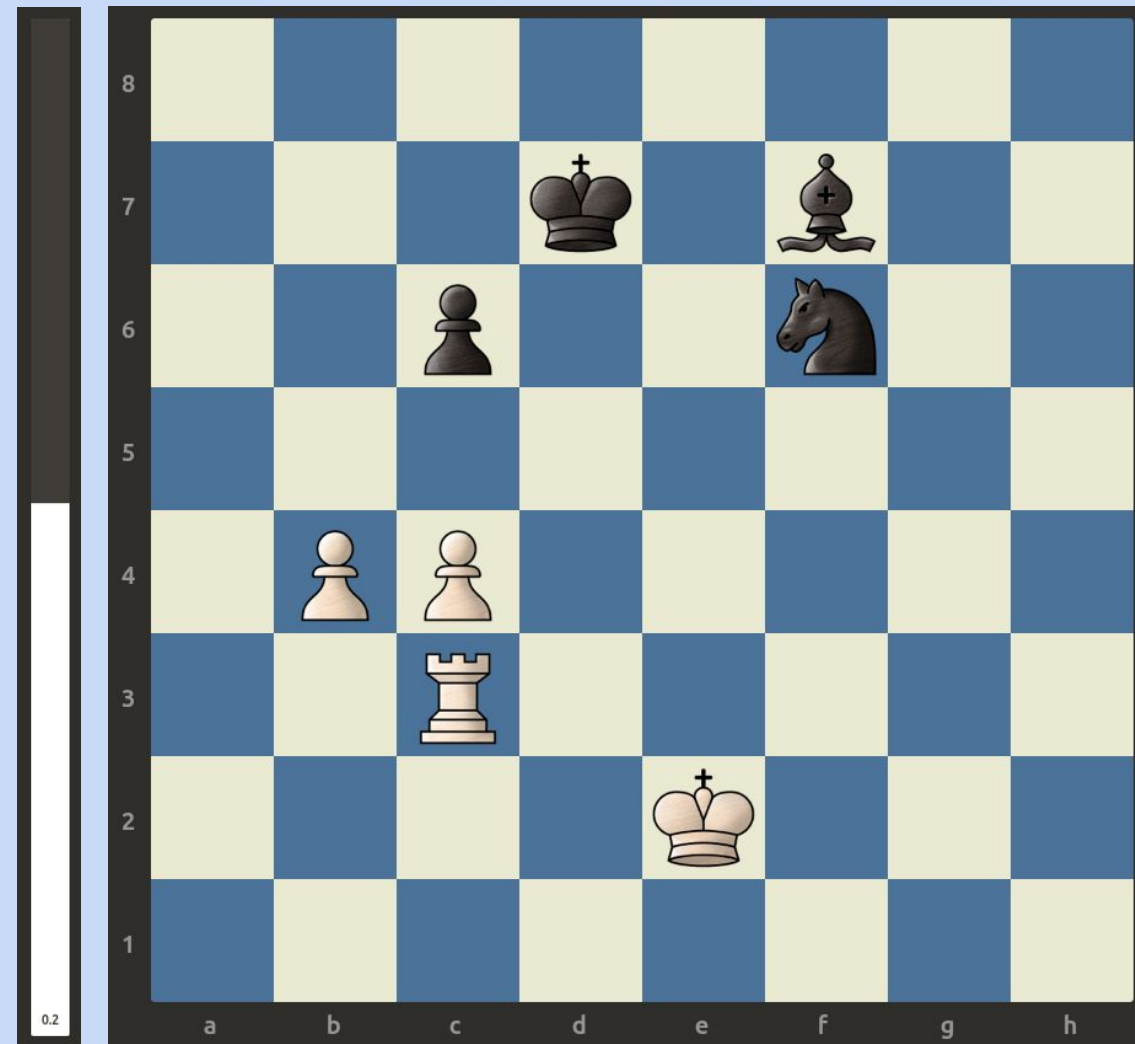
Milhões de posições avaliadas via Stockfish

https://www.wikiwand.com/en/Forsyth%E2%80%9393Edwards_Notation	the position in centipawns. # means that there is forced checkmate.	
12954834 unique values	0	10%
	+13	1%
	Other (11538782)	89%
<code>rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq - 0 1</code>	-10	
<code>rnbqkbnr/pppp1ppp/4p</code>	+56	

Input: Tensor 15x8x8

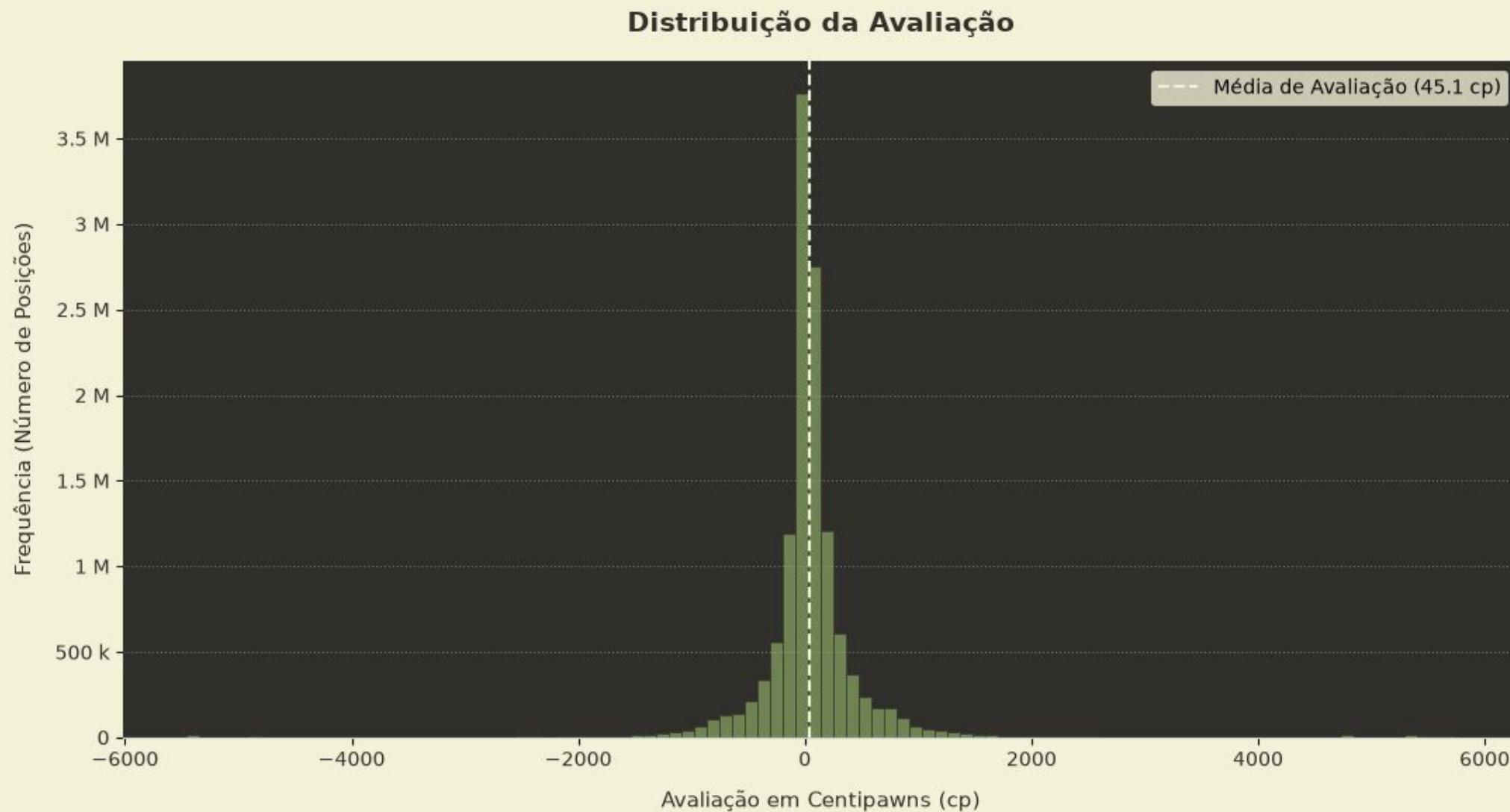
Para o processamento da rede, o estado de FEN é convertido em um tensor tridimensional:

- **12 primeiras Camadas:** Representam os tipos de peças (P, N, B, R, Q, K) para brancas e pretas individualmente.
- **Representação Binária:** Cada casa (8x8) contém 1.0 se a peça estiver presente ou 0.0 caso contrário.
- **Últimas camadas:** A 13ª camada indica a vez de quem joga, e a 14ª e 15ª representam o direito de roque das brancas e das pretas respectivamente.



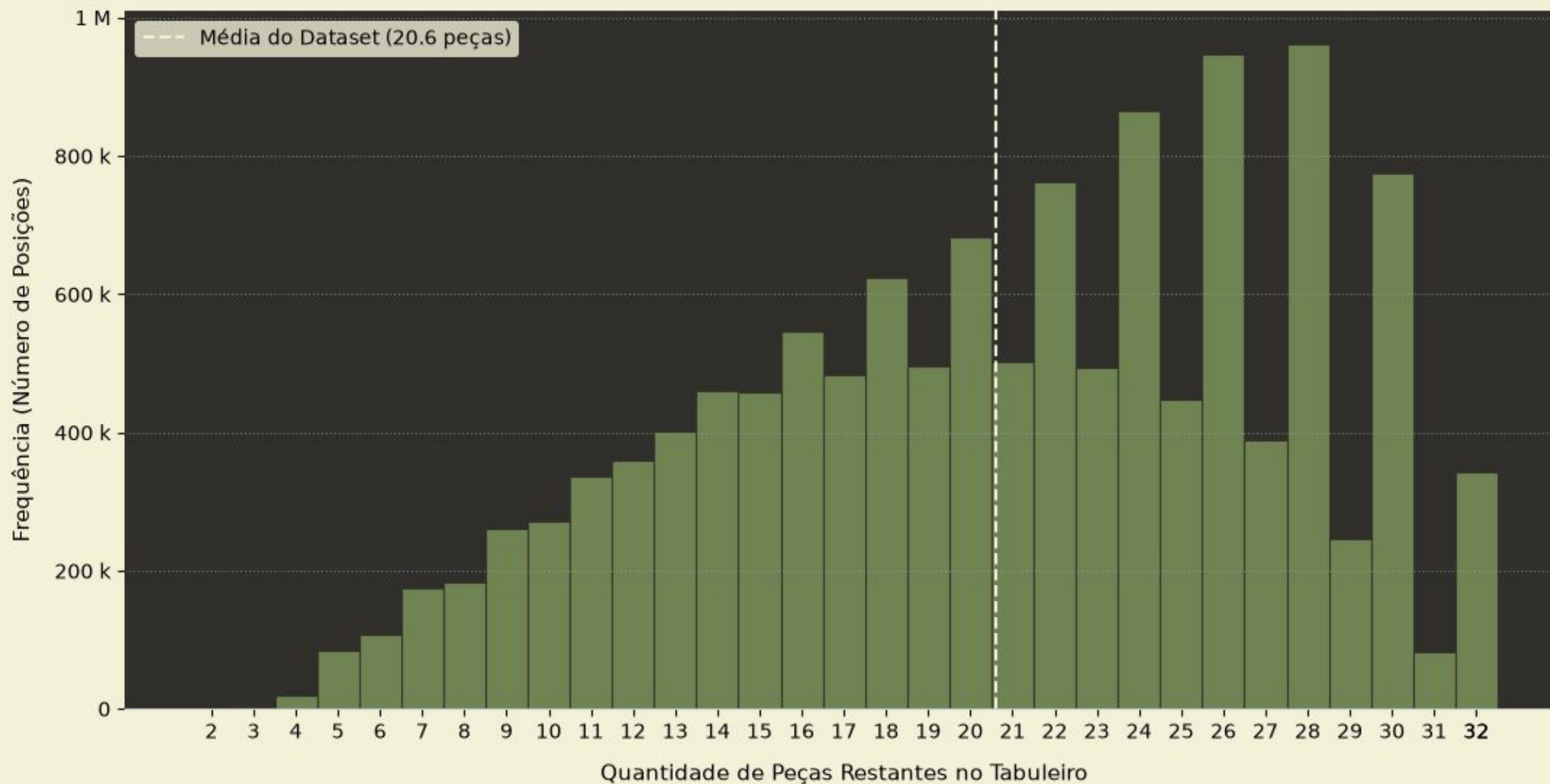
FEN: 8/3k1b2/2p2n2/8/1PP5/2R5/4K3/8 w - - 0 1

Introdução



Introdução

Distribuição da Quantidade de Peças por Posição



CNN (baseline)

Bloco Convolucional:

3 camadas com preservação de resolução espacial (8x8) via padding, permitindo extrair padrões espaciais de curto alcance.

Parâmetros

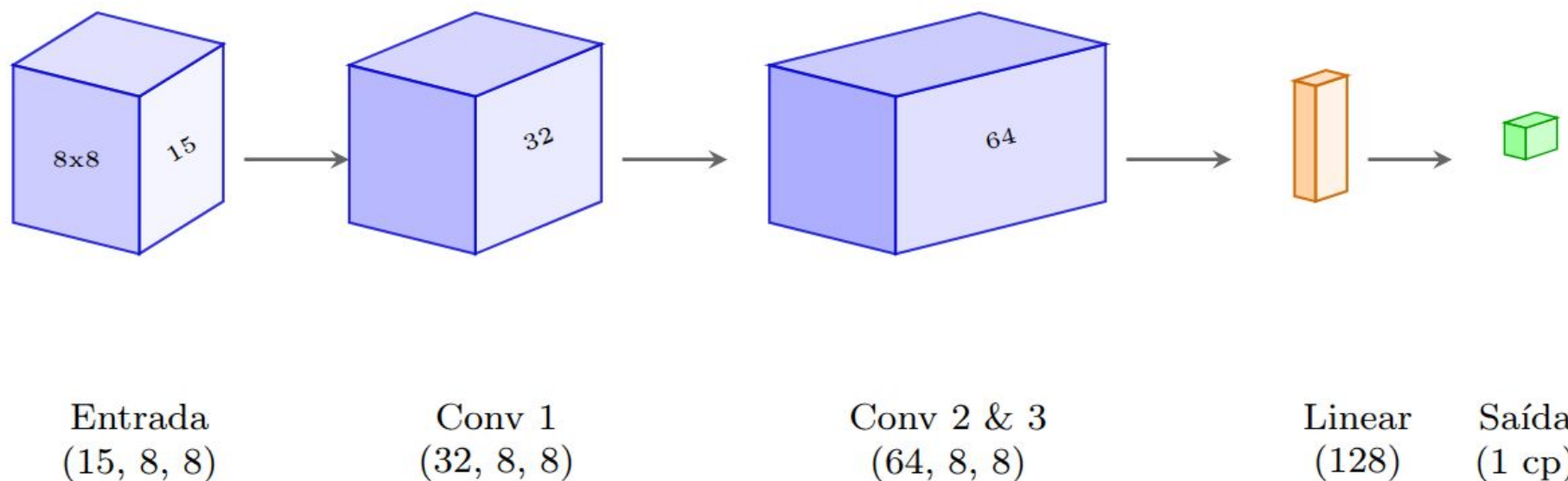
Épocas
30

Learning Rate
0.001

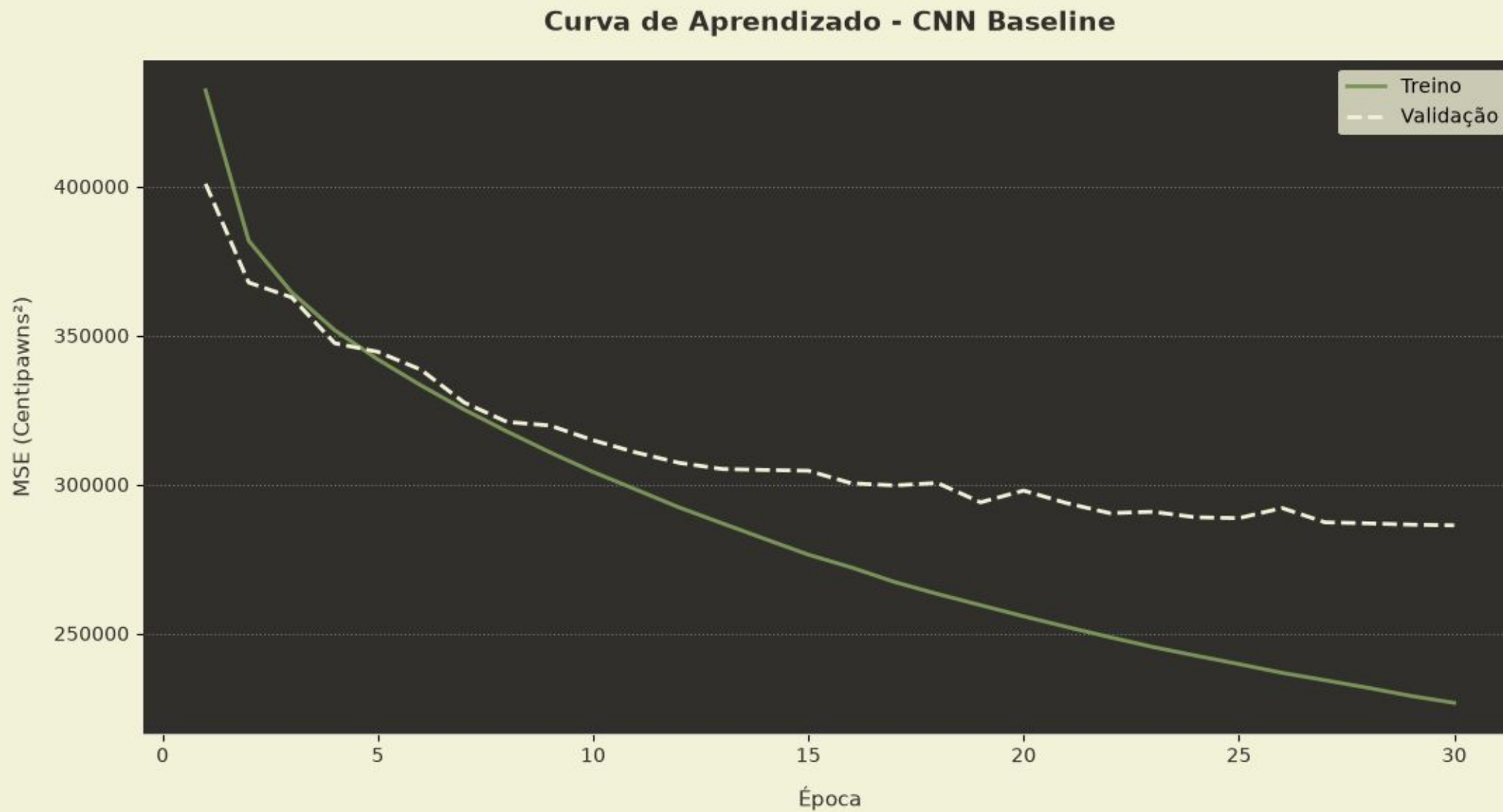
Weight Decay
1e-05

Loss
MSELoss()

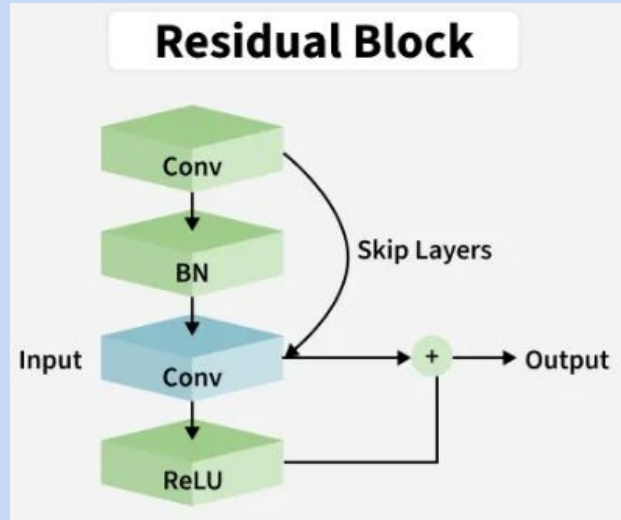
Otimizador
Adam



CNN (baseline)



Resnet



Parâmetros

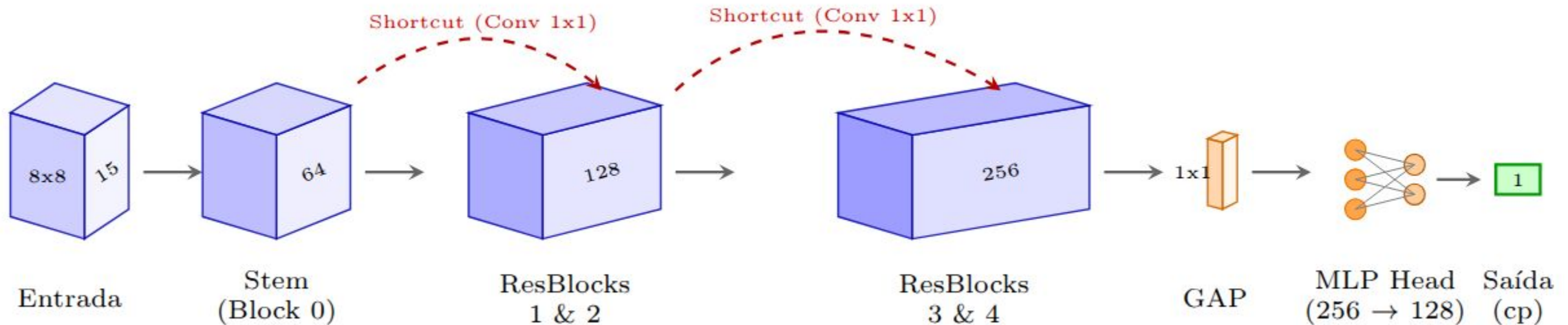
Épocas
30

Learning Rate
0.005

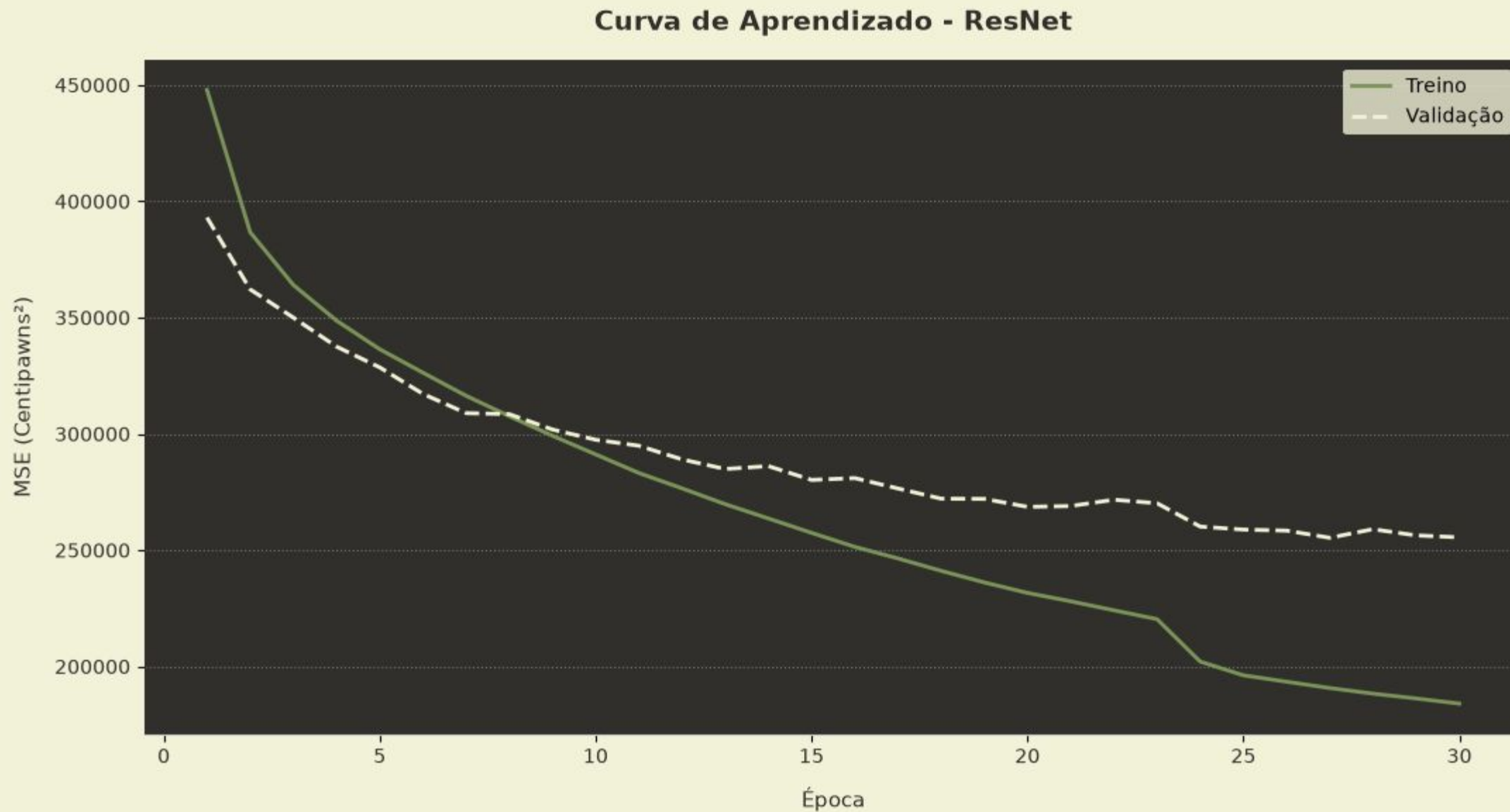
Weight Decay
1e-05

Loss
MSELoss()

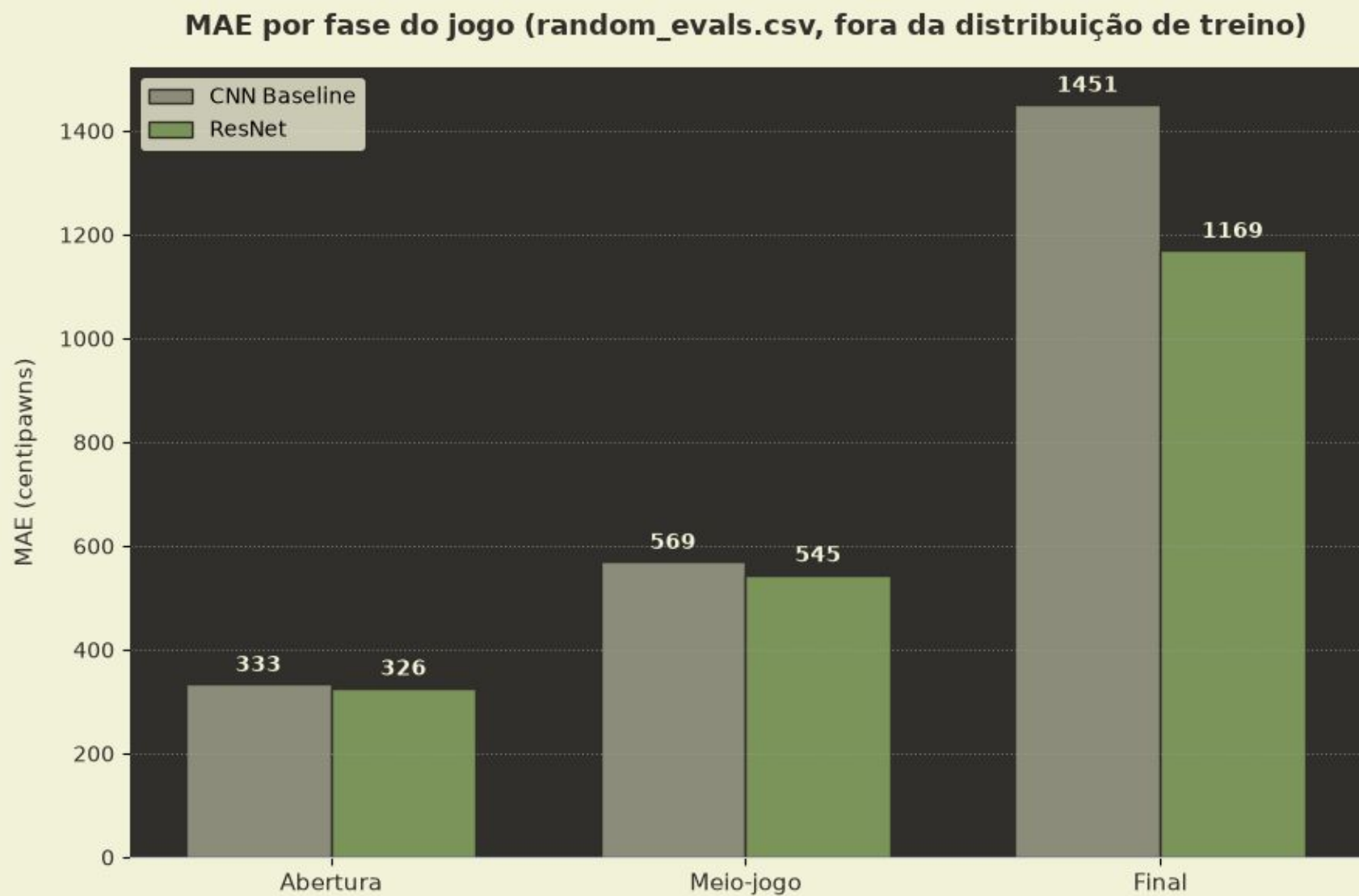
Otimizador
Adam



ResNet



Resultados



Resultados

<https://www.chess.com/analysis/collection/bot-i9jr5MBC/2XVznvBxCN/games>

Daniel_Couto		Bot_Daniel2	
Jogadores			
Precisão	67.3	61.6	
Brilhante	0	!!	0
Excelente	1	!	1
Livro	2	📖	2
Melhor	23	★	18
Ótimo	11	👍	12
Bom	5	✓	6
Imprecisão	3	?!	2
Erro	4	?	7
Chance Perdida	6	✖	5
Capivarada	1	??	2
^			
Rating da Partida	1500	1850	
Abertura	?!	??	
Meio-jogo	?	??	
Final	?	?	

Daniel_Couto		Bot_Daniel2	
Jogadores			
Precisão	72.7	79.4	
Brilhante	0	!!	0
Excelente	1	!	2
Livro	2	📖	2
Melhor	15	★	9
Ótimo	4	👍	7
Bom	2	✓	4
Imprecisão	1	?!	4
Erro	5	?	1
Chance Perdida	1	✖	3
Capivarada	2	??	1
^			
Rating da Partida	1650	2100	
Abertura	?!	??	
Meio-jogo	??	!	
Final	?	✓	

Bot_Daniel3		Daniel_Couto	
Jogadores			
Precisão	71.1	85.4	
Brilhante	0	!!	0
Excelente	2	!	0
Livro	1	📖	1
Melhor	7	★	20
Ótimo	10	👍	5
Bom	2	✓	1
Imprecisão	5	?!	8
Erro	8	?	1
Chance Perdida	2	✖	4
Capivarada	0	??	1
^			
Rating da Partida	1900	2400	
Abertura	?	?	
Meio-jogo	?	?	
Final	?!	!	

Deep Q-Network (DQN)

Aprendizado por Reforço

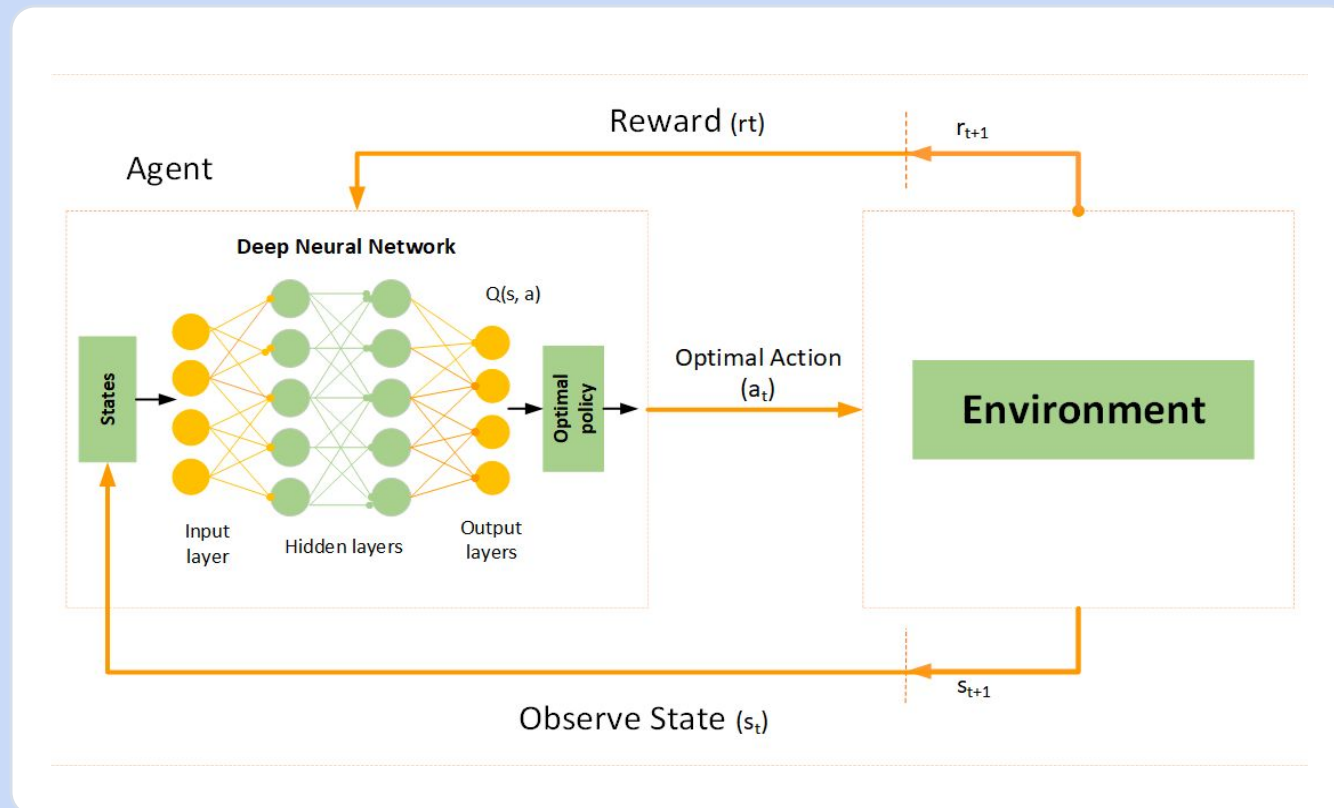
O modelo DQN aprende jogando contra si mesmo (self-play), otimizando a política de lances sem supervisão externa direta.

Replay Buffer

Armazena experiências passadas para quebrar correlações temporais e estabilizar o aprendizado.

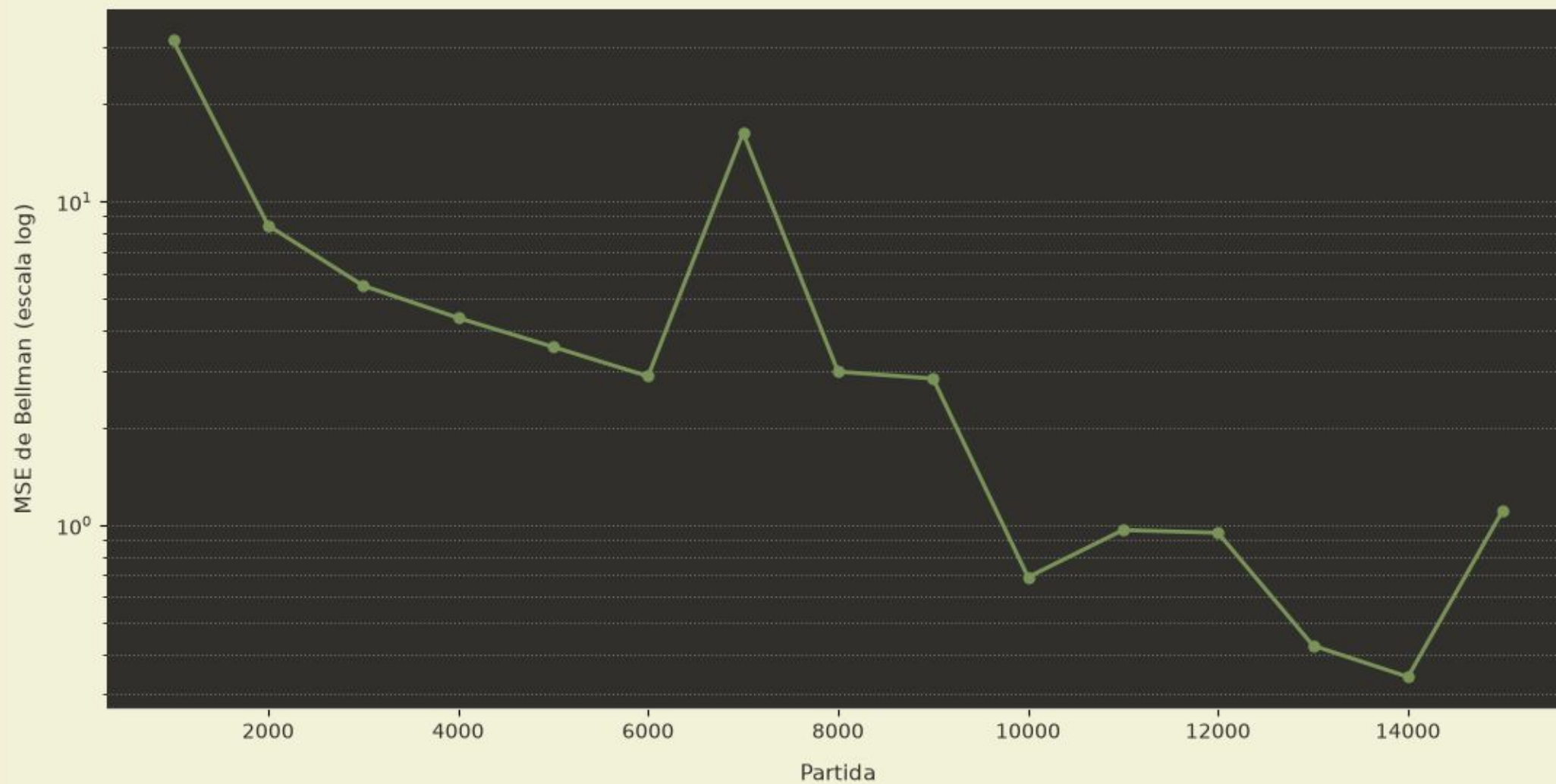
Exploração

Estratégia de decaimento de Épsilon para balancear a descoberta de novas jogadas e o aproveitamento.

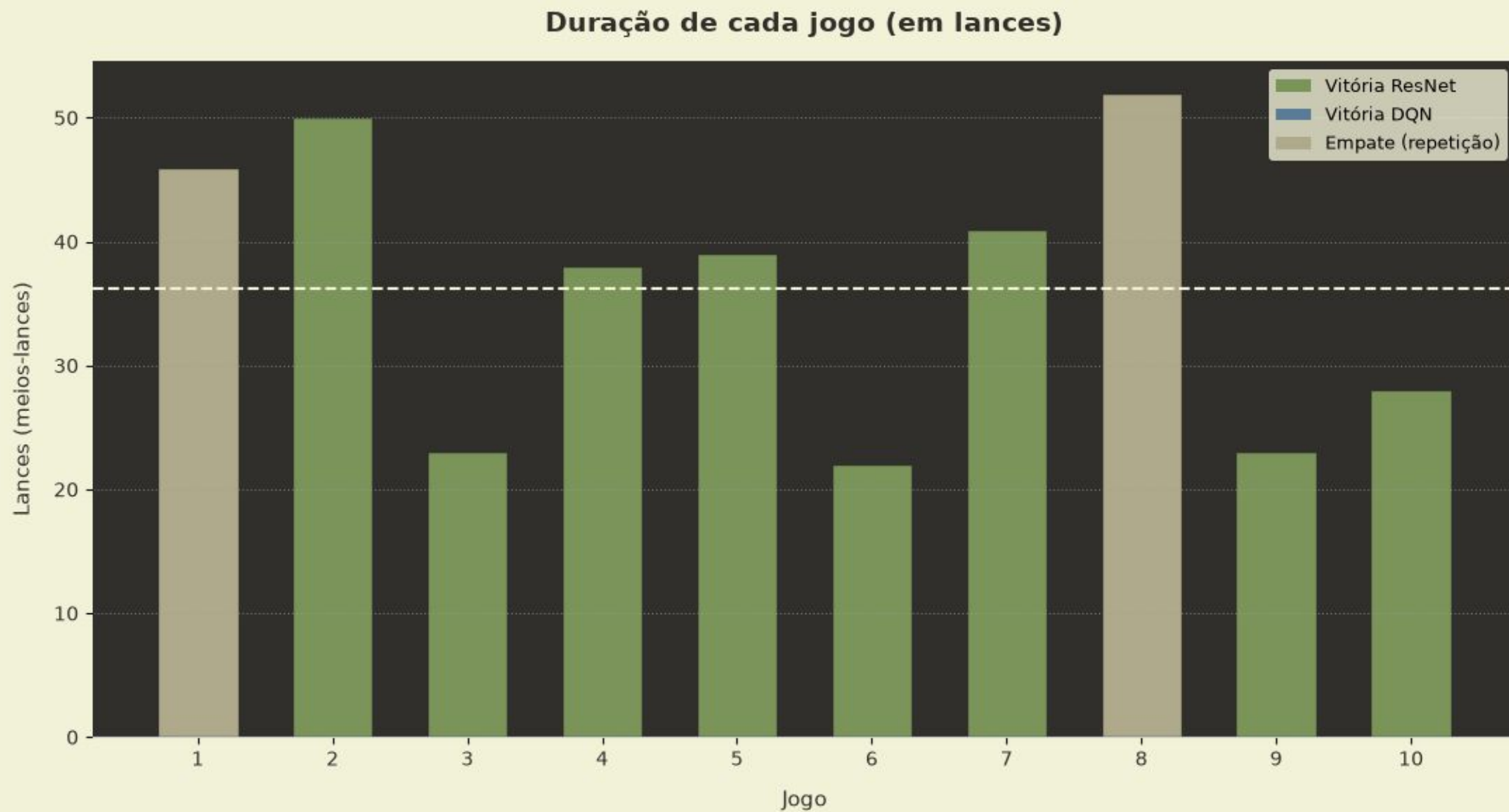


DQN

Loss do DQN ao longo do self-play



Resultados



Referências

- BADHE, Ronak. **Chess Evaluations Dataset**. Kaggle, 2021. Disponível em: <https://www.kaggle.com/datasets/ronakbadhe/chess-evaluations>. Acesso em: 8 jul. 2026.
- HE, Kaiming et al. **Deep residual learning for image recognition**. In: PROCEEDINGS OF THE IEEE CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR). [S. l.: s. n.], 2016. p. 770-778. Disponível em: <https://arxiv.org/abs/1512.03385>. Acesso em: 8 jul. 2026.
- LICHESS ORG. **Lichess Bot**: bridge between lichess API and chess engines. GitHub, 2026. Disponível em: <https://github.com/lichess-bot-devs/lichess-bot>. Acesso em: 8 jul. 2026.
- SILVER, David et al. **Mastering chess and shogi by self-play with a general reinforcement learning algorithm**. Science, [S. l.], v. 362, n. 6419, p. 1140-1144, 2018. Disponível em: <https://arxiv.org/abs/1712.01815>. Acesso em: 8 jul. 2026.

Obrigado!